



Top-Down Parsing (自底向下语法分析)

- ◆ The parse tree is created top to bottom (from root to leaves).
- ◆ By always replacing the leftmost non-terminal symbol via a production rule, we are guaranteed of developing a parse tree in a left-to-right fashion that is consistent with scanning the input.
- ◆ Top-down parser

◆ Recursive-Descent Parsing (递归下降)

- Backtracking is needed (If a choice of a production rule does not work, we backtrack to try other alternatives.)
- It is a general parsing technique, but not widely used.
- Not efficient

$A \Rightarrow aBc \Rightarrow adDc \Rightarrow adec$

◆ Predictive Parsing (可预测) (scan a, scan d, scan e, scan c - accept!)

- no backtracking
- efficient
- needs a special form of grammars (**LL(1) grammars**).
- Recursive Predictive Parsing is a special form of Recursive Descent parsing without backtracking.
- Non-Recursive (Table Driven) Predictive Parser is also known as LL(1) parser.

- ◆ Example:
$$\left. \begin{array}{l} S \rightarrow c A d \\ A \rightarrow ab \mid a \end{array} \right\}$$
 input: cad



Compilers: Principles, Techniques, and Tools

Predictive Parser

a grammar



eliminate
left recursion



left
factor

a grammar suitable for predictive
parsing (a LL(1) grammar)
no %100 guarantee.

- When re-writing a non-terminal in a derivation step, a predictive parser can uniquely choose a production rule by just looking the current symbol in the input string.

current token

$$A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$$

input: ... a



current token

Predictive Parser (example)

$\text{stmt} \rightarrow$ **if** expr **then** stmt **else** stmt
 | **while** expr **do** stmt
 | **begin** stmt_list **end**
 | **for**

- ◆ When we are trying to write the non-terminal *stmt*, if the current token is **if** we have to choose first production rule.
- ◆ When we are trying to write the non-terminal *stmt*, we can uniquely choose the production rule by just looking the current token.
- ◆ We **eliminate the left recursion** in the grammar, and **left factor** it. But it may not be suitable for predictive parsing (not LL(1) grammar).

Recursive Predictive Parsing

Each non-terminal corresponds to a procedure (function).

Ex: $A \rightarrow aBb$ (This is only the production rule for A)

```
proc A {
```

- match the current token with a, and move to the next token;
- call 'B';
- match the current token with b, and move to the next token;

```
}
```

Recursive Predictive Parsing (cont.)

$A \rightarrow aBb \mid bAB$

```
proc A {  
  case of the current token {  
    'a': - match the current token with a, and move to the next token;  
         - call 'B';  
         - match the current token with b, and move to the next token;  
    'b': - match the current token with b, and move to the next token;  
         - call 'A';  
         - call 'B';  
  }  
}
```

Recursive Predictive Parsing (cont.)

- ◆ When to apply ε -productions.

$$A \rightarrow aA \mid bB \mid \varepsilon$$

- ◆ If all other productions fail, we should apply an ε -production. For example, if the current token is not a or b , we may apply the ε -production.
- ◆ Most correct choice: We should apply an ε -production for a non-terminal A when the current token is in the **follow** set of A (which terminals can follow A in the sentential forms).



Recursive Predictive Parsing (Example)

$A \rightarrow aBe \mid cBd \mid C$

$B \rightarrow bB \mid \varepsilon$

$C \rightarrow f$

```
proc A {  
  case of the current token {  
    a: - match the current token with a,  
        and move to the next token;  
        - call B;  
    - match the current token with e,  
        and move to the next token;  
    c: - match the current token with c,  
        and move to the next token;  
        - call B;  
        - match the current token with d,  
        and move to the next token;  
    f: - call C  
  }  
}
```

first set of C

```
proc C { match the current token with f,  
        and move to the next token; }
```

```
proc B {  
  case of the current token {  
    b:- match the current token with b,  
        and move to the next token;  
    - call B  
    e,d: do nothing  
  }  
}
```

follow set of B

Non-Recursive Predictive Parsing – LL(1) Parser

- ◆ Non-Recursive predictive parsing is a **table-driven parser** which has an **input buffer**, a **stack**, a **parsing table**, and an **output stream**.
- ◆ It is also known as LL(1) Parser.

1. Left to right scan input
2. Find leftmost derivation

Grammar: $E \rightarrow TE'$
 $E' \rightarrow +TE' \mid \epsilon$
 $T \rightarrow id$

Input : **id + id \$**
Terminator
↓

Derivation: $E \Rightarrow$

FOLLOW(E') = {\$}

Processing Stack:

LL(1) Parser

input buffer

- our string to be parsed. We will assume that its end is marked with a special symbol \$.

output

- a production rule representing a step of the derivation sequence (left-most derivation) of the string in the input buffer.

stack

← Semantic action

- contains the grammar symbols
- at the bottom of the stack, there is a special end marker symbol \$.
- initially the stack contains only the symbol \$ and the starting symbol S.

\$S ← initial stack

- when the stack is emptied (ie. only \$ left in the stack), the parsing is completed.

parsing table

- a two-dimensional array $M[A,a]$
- each row is a non-terminal symbol
- each column is a terminal symbol or the special symbol \$
- each entry holds a production rule.

Non-Recursive / Table Driven

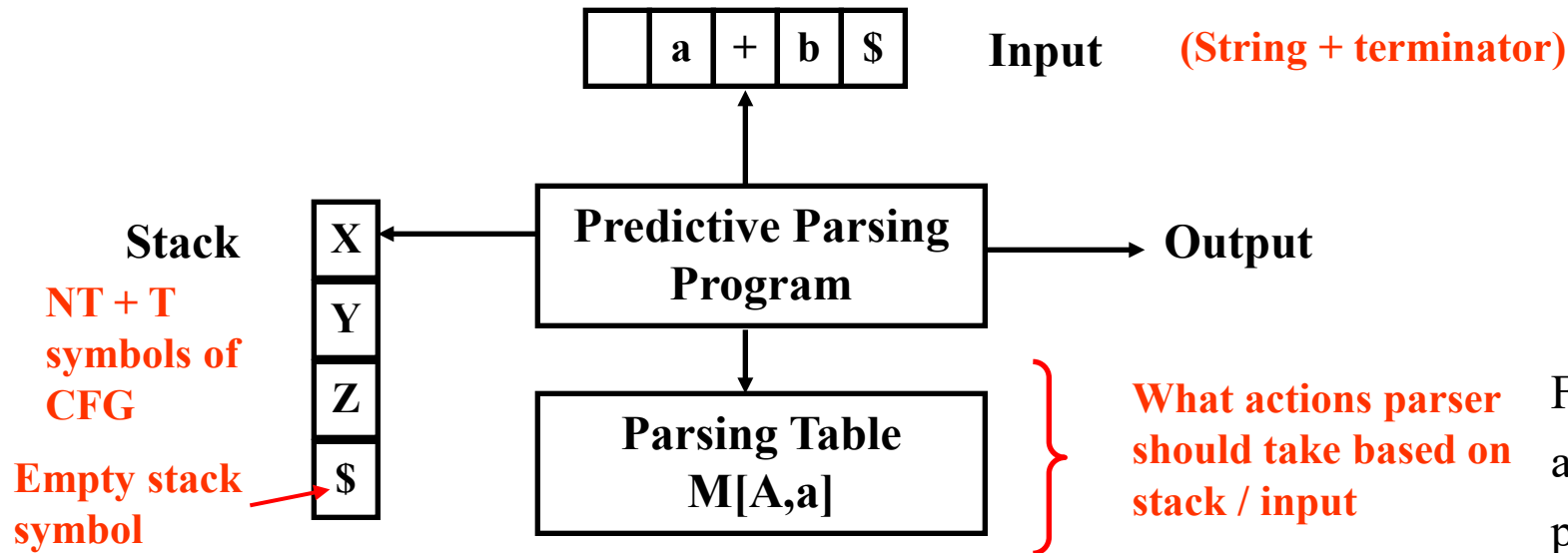


Fig.4.13 model of a non-recursive predictive parser (P.186)

General parser behavior: X : top of stack a : current input

1. When $X=a = \$$ halt, accept, success
2. When $X=a \neq \$$, POP X off stack, advance input, go to 1.
3. When X is a non-terminal, examine $M[X,a]$
 - if it is an error → call recovery routine
 - if $M[X,a] = \{X \rightarrow UVW\}$, POP X , PUSH W,V,U
 - DO NOT expend any input

Notice the pushing order

Algorithm for Non-Recursive Parsing

Set ip to point to the first symbol of $w\$$;

repeat

let X be the top stack symbol and a the symbol pointed to by ip ;

if X is terminal or $\$$ then

if $X=a$ then

pop X from the stack and advance ip

else $error()$

else /* X is a non-terminal */

if $M[X,a] = X \rightarrow Y_1 Y_2 \dots Y_k$ then begin

pop X from stack;

push $Y_k, Y_{k-1}, \dots, Y_1$ onto stack, with Y_1 on top

output the production $X \rightarrow Y_1 Y_2 \dots Y_k$

end

else $error()$

until $X=\$$ /* stack is empty */

 **Input pointer**

 **May also execute other code based on the production used, such as creating parse tree.**

LL(1) Parser – Example1

$S \rightarrow aBa$

$B \rightarrow bB \mid \varepsilon$

	a	b	\$
S	$S \rightarrow aBa$		
B	$B \rightarrow \varepsilon$	$B \rightarrow bB$	

LL(1) Parsing
Table

Sentence “**abba**” is
correct or not

?

stack

input

output

\$S

abba\$

$S \rightarrow aBa$

\$aBa

abba\$

\$aB

bba\$

$B \rightarrow bB$

\$aBb

bba\$

\$aB

ba\$

$B \rightarrow bB$

\$aBb

ba\$

\$aB

a\$

$B \rightarrow \varepsilon$

\$a

a\$

\$

\$

accept, successful completion

LL(1) Parser – Example1 (cont.)

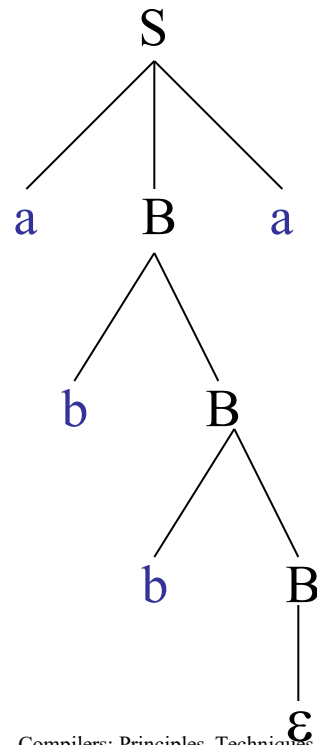
Outputs:

$S \rightarrow aBa$ $B \rightarrow bB$ $B \rightarrow bB$ $B \rightarrow \varepsilon$

Derivation(left-most):

$S \Rightarrow aBa \Rightarrow abBa \Rightarrow abbBa \Rightarrow abba$

parse tree



LL(1) Parser - Example2 (P.188)

Example 4.16:

$E \rightarrow TE'$

$E' \rightarrow + TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow * FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

how to construct *id+id*id* ?

Table M

Non-terminal	INPUT SYMBOL					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow + TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Fig.4.15 parsing table M for grammar

LL(1) Parser – Example2

<u>stack</u>	<u>input</u>	<u>output</u>
\$E	id+id*id\$	
\$E'T	id+id*id\$	$E \rightarrow TE'$
\$E'T'F	id+id*id\$	$T \rightarrow FT'$
\$E'T'id	id+id*id\$	$F \rightarrow id$
\$E'T'	+id*id\$	
\$E'	+id*id\$	$T' \rightarrow \epsilon$
\$E'T+	+id*id\$	$E' \rightarrow +TE'$
\$E'T	id*id\$	
\$E'T'F	id*id\$	$T \rightarrow FT'$
\$E'T'id	id*id\$	$F \rightarrow id$
\$E'T'	*id\$	
\$E'T'F*	*id\$	$T' \rightarrow *FT'$
\$E'T'F	id\$	
\$E'T'id	id\$	$F \rightarrow id$
\$E'T'	\$	
\$E'	\$	$T' \rightarrow \epsilon$
\$	\$	$E' \rightarrow \epsilon$ accept

Fig.4.16 moves made by predictive parser on input id+id*id.

Leftmost Derivation for the Example

The leftmost derivation for the example is as follows:

$$E \Rightarrow TE'$$

$$\Rightarrow FT'E'$$

$$\Rightarrow id\ T'E'$$

$$\Rightarrow id\ E'$$

$$\Rightarrow id + TE'$$

$$\Rightarrow id + FT'E'$$

$$\Rightarrow id + id\ T'E'$$

$$\Rightarrow id + id * FT'E'$$

$$\Rightarrow id + id * id\ T'E'$$

$$\Rightarrow id + id * id\ E'$$

$$\Rightarrow id + id * id$$

Constructing LL(1) Parsing Tables

Constructing the Parsing Table M !

1st : Calculate FIRST & FOLLOW for Grammar

2nd: Apply Construction Algorithm for Parsing Table
(We'll see this shortly)

Basic Functions:

FIRST: Let α be a string of grammar symbols. FIRST(α) is the set that includes every terminal that appears leftmost in α or in any string originating from α .

NOTE: If $\alpha \Rightarrow \epsilon$, then ϵ is FIRST(α).

FOLLOW: Let A be a non-terminal. FOLLOW(A) is the set of terminals a that can appear directly to the right of A in some sentential form. ($S \Rightarrow \alpha A a \beta$, for some α and β).

NOTE: If $S \Rightarrow \alpha A$, then $\$$ is FOLLOW(A).

Compute FIRST for Any String X

1. If X is a terminal, $\text{FIRST}(X) = \{X\}$
2. If $X \rightarrow \epsilon$ is a production rule, add ϵ to $\text{FIRST}(X)$
3. If X is a non-terminal, and $X \rightarrow Y_1 Y_2 \dots Y_k$ is a production rule

Place $\text{FIRST}(Y_1)$ in $\text{FIRST}(X)$

if $Y_1 \Rightarrow \epsilon$, Place $\text{FIRST}(Y_2)$ in $\text{FIRST}(X)$

if $Y_2 \Rightarrow \epsilon$, Place $\text{FIRST}(Y_3)$ in $\text{FIRST}(X)$

...

if $Y_{k-1} \Rightarrow \epsilon$, Place $\text{FIRST}(Y_k)$ in $\text{FIRST}(X)$

NOTE: As soon as $Y_i \Rightarrow \epsilon$, Stop

Repeat above steps until no more elements are added to any $\text{FIRST}()$ set.

Checking “ $Y_i \Rightarrow \epsilon$?” essentially amounts to checking whether ϵ belongs to $\text{FIRST}(Y_i)$

Computing FIRST(X) :

All Grammar Symbols - continued

Informally, suppose we want to compute

$$\text{FIRST}(X_1 X_2 \dots X_n) = \text{FIRST}(X_1)$$

“+”FIRST(X_2) if ϵ is in FIRST(X_1)

“+”FIRST(X_3) if ϵ is in FIRST(X_2)

...

“+”FIRST(X_n) if ϵ is in FIRST(X_{n-1})

Note 1: Only add ϵ to FIRST($X_1 X_2 \dots X_n$) if ϵ is in FIRST(X_i) for all i

Note 2: For FIRST(X_1), if $X_1 \rightarrow Z_1 Z_2 \dots Z_m$, then we need to compute FIRST($Z_1 Z_2 \dots Z_m$) !

FIRST Example

Example 4.17:

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \quad | \quad \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \quad | \quad \varepsilon$$
$$F \rightarrow (E) \quad | \quad id$$
$$\text{FIRST}(F) = \{ (, id \}$$
$$\text{FIRST}(T') = \{ *, \varepsilon \}$$
$$\text{FIRST}(T) = \{ (, id \}$$
$$\text{FIRST}(E') = \{ +, \varepsilon \}$$
$$\text{FIRST}(E) = \{ (, id \}$$
$$\text{FIRST}(TE') = \{ (, id \}$$
$$\text{FIRST}(+TE') = \{ + \}$$
$$\text{FIRST}(\varepsilon) = \{ \varepsilon \}$$
$$\text{FIRST}(FT') = \{ (, id \}$$
$$\text{FIRST}(*FT') = \{ * \}$$
$$\text{FIRST}(\varepsilon) = \{ \varepsilon \}$$
$$\text{FIRST}((E)) = \{ (\}$$
$$\text{FIRST}(id) = \{ id \}$$

Alternative way to compute FIRST

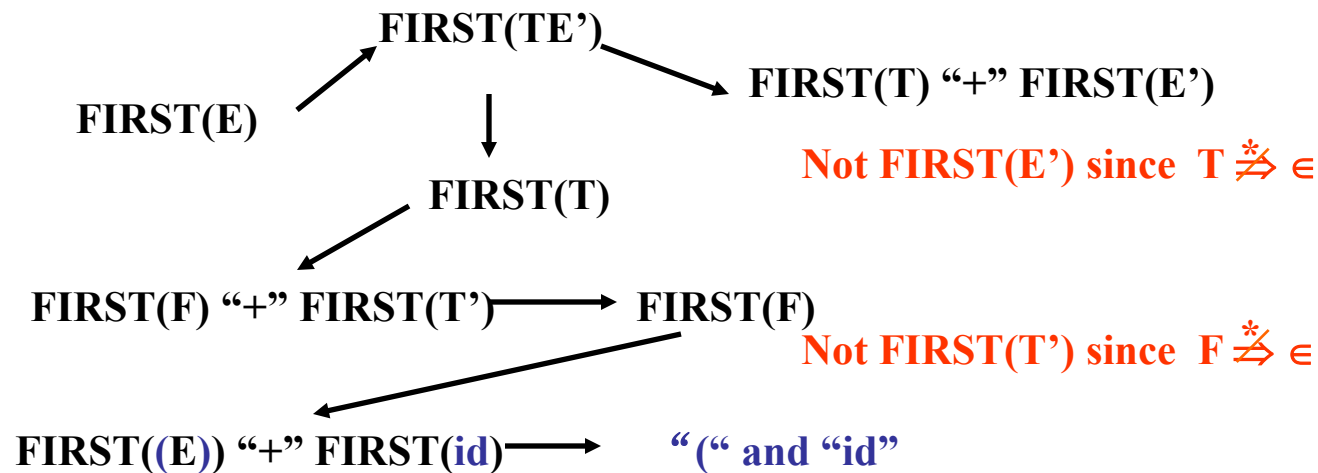
Computing FIRST for:

$$E \rightarrow TE'$$

$$E' \rightarrow + TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow * FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$


Overall: $FIRST(E) = FIRST(T) = FIRST(F) = \{ (, id \}$

$FIRST(E') = \{ +, \epsilon \}$ $FIRST(T') = \{ *, \epsilon \}$

Compute FOLLOW (for non-terminals)

1. If S is the start symbol $\rightarrow$ $\$$ is in FOLLOW(S)

Initially $\$S$

2. If $A \rightarrow \alpha B \beta$ is a production rule $\rightarrow$ everything in FIRST(β) is FOLLOW(B) except ϵ

3. If ($A \rightarrow \alpha B$ is a production rule) or

($A \rightarrow \alpha B \beta$ is a production rule and ϵ is in FIRST(β))

$\rightarrow$ everything in FOLLOW(A) is in FOLLOW(B).

(Whatever followed A must follow B , since nothing follows B from the production rule)

We apply these rules until nothing more can be added to any FOLLOW set.

The Algorithm for FOLLOW – pseudocode

1. Initialize FOLLOW(X) for all non-terminals X to empty set. Place \$ in FOLLOW(S), where S is the start NT.
2. Repeat the following step until no modifications are made to any Follow-set

For any production $X \rightarrow X_1 X_2 \dots X_j X_{j+1} \dots X_m$

For $j=1$ to m ,

if X_j is a non-terminal then:

$\text{FOLLOW}(X_j) = \text{FOLLOW}(X_j) \cup (\text{FIRST}(X_{j+1}, \dots, X_m) - \{\epsilon\});$

If $\text{FIRST}(X_{j+1}, \dots, X_m)$ contains ϵ or $X_{j+1}, \dots, X_m = \epsilon$

then $\text{FOLLOW}(X_j) = \text{FOLLOW}(X_j) \cup \text{FOLLOW}(X);$

FOLLOW Example

Example 4.17:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id}$$

$$\text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(E) = \{ (, \text{id} \}$$

$$\text{FOLLOW}(E) = \{), \$ \}$$

$$\text{FOLLOW}(E') = \{), \$ \}$$

$$\text{FOLLOW}(T) = \{ +,), \$ \}$$

$$\text{FOLLOW}(T') = \{ +,), \$ \}$$

$$\text{FOLLOW}(F) = \{ *, +,), \$ \}$$

Motivation Behind FIRST & FOLLOW



FIRST: Is used to help find the appropriate reduction to follow given the top-of-the-stack non-terminal and the current input symbol.

Example: If $A \rightarrow \alpha$, and a is in $\text{FIRST}(\alpha)$, then when $a = \text{input}$, replace A with α (in the stack).

(a is one of first symbols of α , so when A is on the stack and a is input, **POP** A and **PUSH** α .)

FOLLOW: Is used when FIRST has a conflict, to resolve choices, or when FIRST gives no suggestion. When $\alpha \rightarrow \epsilon$ or $\alpha \Rightarrow^* \epsilon$, then what follows A dictates the next choice to be made.

Example: If $A \rightarrow \alpha$, and b is in $\text{FOLLOW}(A)$, then when $\alpha \Rightarrow^* \epsilon$ and b is an input character, then we expand A with α , which will eventually expand to ϵ , of which b follows!

($\alpha \Rightarrow^* \epsilon$: i.e., **FIRST**(α) contains ϵ .)

Exercises

◆ Given the following grammar:

$$S \rightarrow aAb \mid Sc \mid \varepsilon$$

$$A \rightarrow aAb \mid \varepsilon$$

◆ Please compute $\text{FIRST}()$ and $\text{FOLLOW}()$ for each non-terminal.

$$S \rightarrow aAb \mid Sc \mid \varepsilon$$

$$A \rightarrow aAb \mid \varepsilon$$

Eliminating left-recursive

$$S \rightarrow aAbS' \mid S'$$

$$S' \rightarrow cS' \mid \varepsilon$$

$$A \rightarrow aAb \mid \varepsilon$$

$$\text{FIRST}(S) = \{a, c, \varepsilon\}$$

$$\text{FIRST}(S') = \{c, \varepsilon\}$$

$$\text{FIRST}(A) = \{a, \varepsilon\}$$

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(S') = \{\$ \}$$

$$\text{FOLLOW}(A) = \{b\}$$

Constructing LL(1) Parsing Table

Algorithm 4.4:

1. Repeat Steps 2 & 3 for each rule $A \rightarrow \alpha$
2. Terminal a in $\text{FIRST}(\alpha)$? Add $A \rightarrow \alpha$ to $M[A, a]$
- 3.1 ϵ in $\text{FIRST}(\alpha)$? Add $A \rightarrow \alpha$ to $M[A, b]$ for all terminals b in $\text{FOLLOW}(A)$.
- 3.2 ϵ in $\text{FIRST}(\alpha)$ and $\$$ in $\text{FOLLOW}(A)$? Add $A \rightarrow \alpha$ to $M[A, \$]$
4. All undefined entries are errors.



Constructing LL(1) Parsing Table – Example

Example 4.18:

$E \rightarrow TE'$ $FIRST(TE') = \{ (, id \}$ $\rightarrow E \rightarrow TE'$ into $M[E, (]$ and $M[E, id]$

$E' \rightarrow +TE'$ $FIRST(+TE') = \{ + \}$ $\rightarrow E' \rightarrow +TE'$ into $M[E', +]$

$E' \rightarrow \epsilon$ $FIRST(\epsilon) = \{ \epsilon \}$ $\rightarrow$ none
but since ϵ in $FIRST(\epsilon)$
and $FOLLOW(E') = \{ \$,) \}$ $\rightarrow E' \rightarrow \epsilon$ into $M[E', \$]$ and $M[E',)]$

$T \rightarrow FT'$ $FIRST(FT') = \{ (, id \}$ $\rightarrow T \rightarrow FT'$ into $M[T, (]$ and $M[T, id]$

$T' \rightarrow *FT'$ $FIRST(*FT') = \{ * \}$ $\rightarrow T' \rightarrow *FT'$ into $M[T', *]$

$T' \rightarrow \epsilon$ $FIRST(\epsilon) = \{ \epsilon \}$ $\rightarrow$ none
but since ϵ in $FIRST(\epsilon)$
and $FOLLOW(T') = \{ \$,), + \}$ $\rightarrow T' \rightarrow \epsilon$ into $M[T', \$]$, $M[T',)]$ and $M[T', +]$

$F \rightarrow (E)$ $FIRST((E)) = \{ (\}$ $\rightarrow F \rightarrow (E)$ into $M[F, (]$

$F \rightarrow id$ $FIRST(id) = \{ id \}$ $\rightarrow F \rightarrow id$ into $M[F, id]$

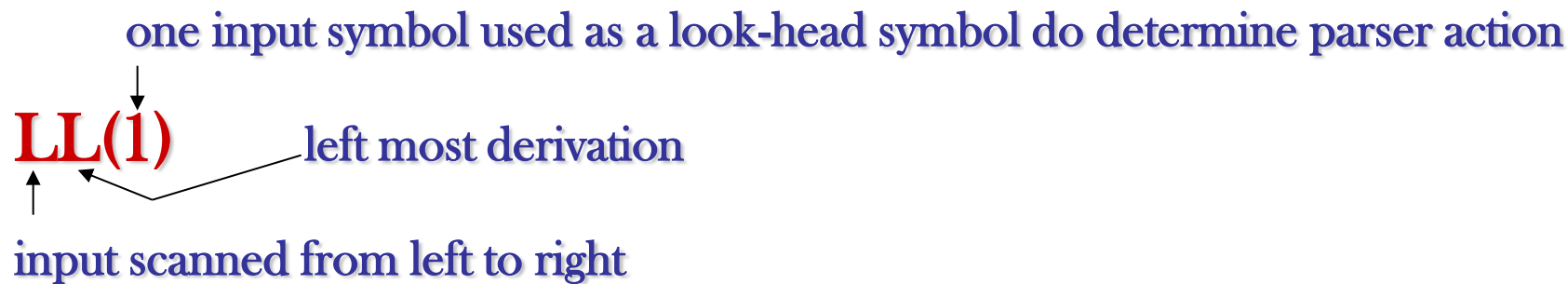
Constructing LL(1) Parsing Table (cont.)

Non-terminal	Input symbol					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Fig.4.15 parsing table M for grammar

LL(1) Grammars (P.191)

- ◆ A grammar whose parsing table has no multiply-defined entries is said to be LL(1) grammar.



- ◆ The parsing table of a grammar may contain more than one production rule (namely multiply-defined entry). In this case we say that it is not a LL(1) grammar, such as left recursive or ambiguous.

L : Scan input from Left to Right

L : Construct a Leftmost Derivation

1 : Use “1” input symbol as lookahead in conjunction with stack to decide on the parsing action

LL(1) grammars == they have no multiply-defined entries in the parsing table.

Properties of LL(1) grammars:

- Grammar can't be ambiguous or left recursive
- Grammar is LL(1) $\Leftrightarrow$ when $A \rightarrow \alpha \mid \beta$
 1. α & β do not derive strings starting with the same terminal a
 2. Either α or β can derive ϵ , but not both.

Note: It may not be possible for a grammar to be manipulated into an LL(1) grammar

Properties of LL(1) Grammars

A grammar G is LL(1) if and only if the following conditions hold for two distinctive production rules $A \rightarrow \alpha$ and $A \rightarrow \beta$

1. Both α and β cannot derive strings starting with same terminals.
2. At most one of α and β can derive to ϵ .
3. If β can derive to ϵ , then α cannot derive to any string starting with a terminal in FOLLOW(A).

A Grammar which is not LL(1)

Example:

$S \rightarrow i C t S E \mid a$

$E \rightarrow e S \mid \varepsilon$

$C \rightarrow b$

$\text{FIRST}(iCtSE) = \{i\}$

$\text{FIRST}(a) = \{a\}$

$\text{FIRST}(eS) = \{e\}$

$\text{FIRST}(\varepsilon) = \{\varepsilon\}$

$\text{FIRST}(b) = \{b\}$

$\text{FOLLOW}(S) = \{ \$, e \}$

$\text{FOLLOW}(E) = \{ \$, e \}$

$\text{FOLLOW}(C) = \{ t \}$

	a	b	e	i	t	\$
S	$S \rightarrow a$			$S \rightarrow iCtSE$		
E			$E \rightarrow e S$ $E \rightarrow \varepsilon$			$E \rightarrow \varepsilon$
C		$C \rightarrow b$				

Problem $\rightarrow$ ambiguity

two production rules for $M[E,e]$

Exercise in class

◆ Grammar:

$\text{lexp} \rightarrow \text{atom} \mid \text{list}$

$\text{atom} \rightarrow \text{num} \mid \text{id}$

$\text{list} \rightarrow (\text{lexp_seq})$

$\text{lexp_seq} \rightarrow \text{lexp_seq lexp} \mid \text{lexp}$

Please calculate $\text{FIRST}()$, $\text{FOLLOW}()$,then construct parsing table and see whether the grammar is LL(1).

消除左递归

$\text{lexp} \rightarrow \text{atom} \mid \text{list}$

$\text{atom} \rightarrow \text{numr} \mid \text{id}$

$\text{list} \rightarrow (\text{lexp-seq})$

$\text{lexp-seq} \rightarrow \text{lexp} \text{lexp-seq}'$

$\text{lexp-seq}' \rightarrow \text{lexp} \text{lexp-seq}' \mid \varepsilon$

$\text{lexp} \rightarrow \text{atom} \mid \text{list}$

$\text{atom} \rightarrow \text{num} \mid \text{id}$

$\text{list} \rightarrow (\text{lexp-seq})$

$\text{lexp-seq} \rightarrow \text{lexp lexp-seq}'$

$\text{lexp-seq}' \rightarrow \text{lexp lexp-seq}' \mid \epsilon$

$\text{FIRST}(\text{lexp}) = \{ (, \text{num}, \text{id} \}$

$\text{FIRST}(\text{atom}) = \{ \text{num}, \text{id} \}$

$\text{FIRST}(\text{list}) = \{ (\}$

$\text{FIRST}(\text{lexp-seq}) = \{ (, \text{num}, \text{id} \}$

$\text{FIRST}(\text{lexpseq}') = \{ (, \text{num}, \text{id}, \epsilon \}$

$\text{FOLLOW}(\text{lexp-seq}') = \{ \}$

- 1 $\text{lexp} \rightarrow \text{atom}$
- 2 $\text{lexp} \rightarrow \text{list}$
- 3 $\text{atom} \rightarrow \text{num}$
- 4 $\text{atom} \rightarrow \text{id}$

- 5 $\text{list} \rightarrow (\text{lexp_seq})$
- 6 $\text{lexp_seq} \rightarrow \text{lexp lexp_seq}'$
- 7 $\text{lexp_seq}' \rightarrow \text{lexp lexp_seq}'$
- 8 $\text{lexp_seq}' \rightarrow \varepsilon$

	num	id	(	)	\$
lexp	1	1	2		
atom	3	4			
list			5		
lexp_seq	6	6	6		
lexp_seq'	7	7	7	8	

因为表项没有冲突，所以该文法是LL(1)文法。

Final Comments – Top-Down Parsing

So far,

- ◆ We've examined grammars and language theory and its relationship to parsing
- ◆ Key concepts: Rewriting grammar into an acceptable form

Examined Top-Down parsing:

- ◆ Brute Force : Transition diagrams & recursion
- ◆ Elegant : Table driven

We've identified its shortcomings:

- ◆ Not all grammars can be made LL(1) !
- ◆ Bottom-Up Parsing - Future

小结

- ◆理解递归下降的语法分析方法
- ◆理解递归可预测的语法分析方法
- ◆掌握非递归可预测的语法分析方法，即LL(1)语法分析法
- ◆掌握计算FIRST 和FOLLOW的算法
- ◆掌握LL(1)语法分析表的构造